Lock-Free Parallel SGD on Multicore Systems
Project Report

Ethan Luvisia and Leo Liang

April 24, 2026

1. Project Goals

The main goal of this project was to test a practical systems question: **when does Hogwild-style lock-free SGD actually beat synchronized alternatives on a real multicore CPU?** Instead of assuming lock-free is always better, we wanted to measure where each approach wins and loses.

We implemented and compared the following training modes in a reproducible framework:

- `serial`: one thread, no synchronization needed
- `coarse_lock`: one global lock around updates
- `striped_lock`: lock striping over parameter partitions
- `hogwild`: mutex-free asynchronous updates using atomic/CAS coordinate operations
- `local_batch_reduce` (optional baseline for MF): local accumulation then periodic reduction

Conceptually, we followed the `hogwildTR.pdf` framing: sparse updates with low overlap should favor lock-free methods, while dense or highly contended updates should reduce those gains.

2. How Experimentation Was Done

2.1 Workloads

We used two synthetic workloads to study different sparsity/overlap behavior:

- **Sparse Logistic Regression**: configurable sample count N, dimension d, and active features per update k.

- **Matrix Factorization (MF)**: configurable users/items/rank/observations; each update touches one user and one item factor vector.

2.2 Fairness and Reproducibility Controls

To keep comparisons meaningful, runs shared:

- same dataset generation seed,
- same initialization seed,
- same epoch/update-touch budget,
- same logging cadence and evaluation checkpoints,
- same target-loss definition per workload,
- trial IDs for repeatability and median-based analysis.

This means performance differences are mostly from synchronization behavior and thread interaction, not from changing data or easier initializations.

2.3 Runtime Environment

Reported full-suite runs were on a 12-thread macOS arm64 machine (OpenMP-enabled GCC 15 build), with output recorded in `results_full/summary.csv` and `results_full/trace.csv`. Plots were generated by the project scripts into `report/figures/`.

3. Graph-by-Graph Results and Comparison

This section includes every generated report figure and explains what it reveals.

3.1 Logistic: Time-to-Target vs Threads

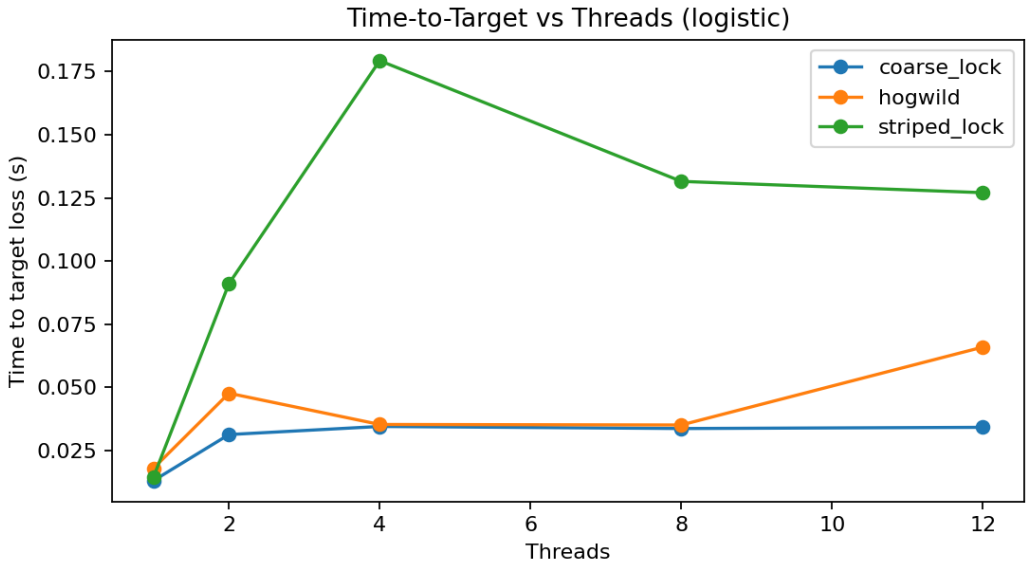

Figure 1: Logistic workload: time-to-target-loss vs thread count.

Interpretation: increasing threads did not consistently reduce time-to-target for this lightweight logistic setup. A key reason is that total runtime is already very short, so synchronization/parallel overhead becomes a large fraction of execution.

Comparison takeaway: `hogwild` is usually better than heavy locking at moderate thread counts, but `serial` can still be fastest when the per-run compute is too small.

3.2 Logistic: Throughput vs Threads

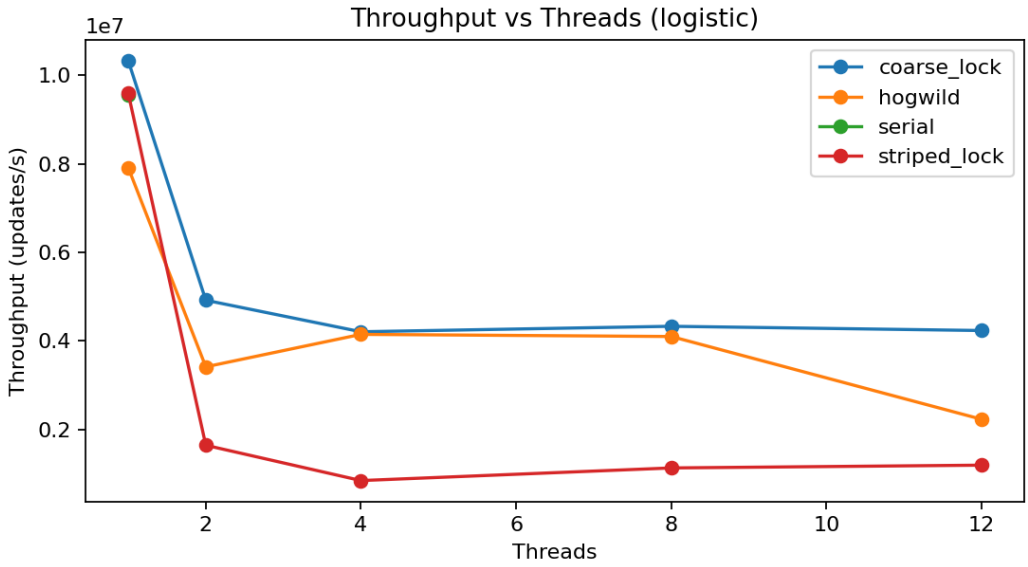

Figure 2: Logistic workload: update throughput vs thread count.

Interpretation: throughput rises with more threads up to a point, but eventually flattens or degrades due to memory traffic and contention effects.

Comparison takeaway: lock-free updates can deliver higher throughput than synchronized baselines in low-overlap regions, but high thread counts do not guarantee linear scaling.

3.3 Logistic: Speedup vs Threads

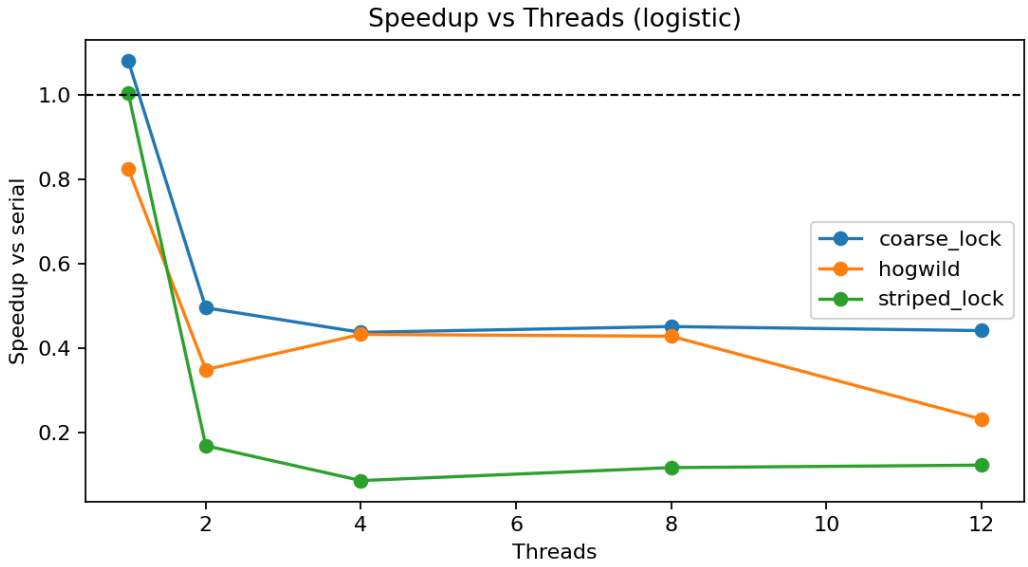

Figure 3: Logistic workload: speedup relative to serial baseline.

Interpretation: speedup values are often below 1 for this configuration, meaning parallel overhead dominates potential gains.

Comparison takeaway: on small/fast logistic runs, `serial` remains difficult to beat; the lock-free advantage is real but not enough to always overcome overhead.

3.4 Logistic: Loss vs Time

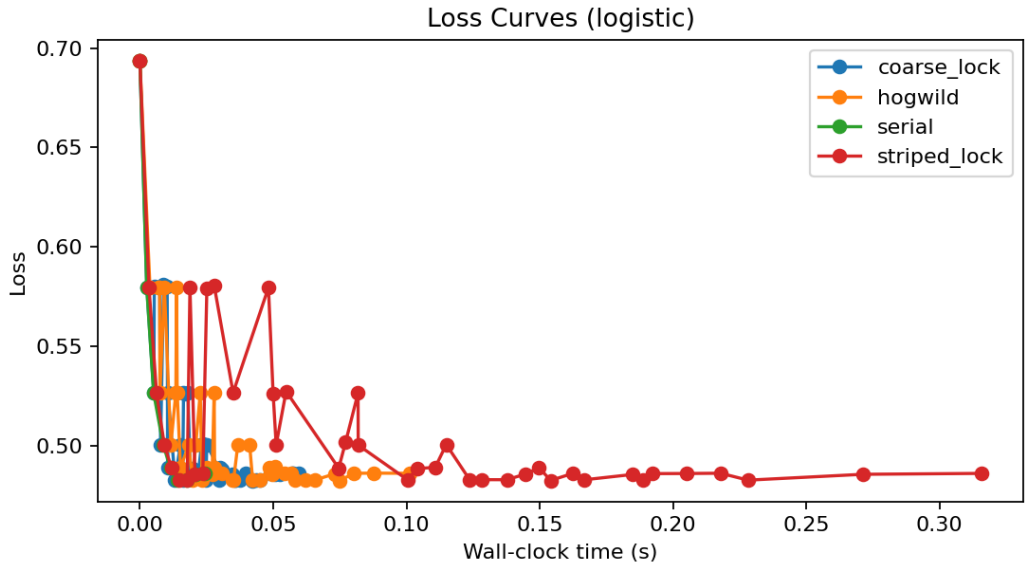

Figure 4: Logistic workload: convergence trajectories over wall-clock time.

Interpretation: all methods can reduce loss, but wall-clock convergence differs by scheduling and synchronization cost.

Comparison takeaway: `hogwild` often reaches competitive loss quickly in sparse regimes, while lock-based methods may pay extra latency from synchronization.

3.5 Logistic: Loss vs Updates

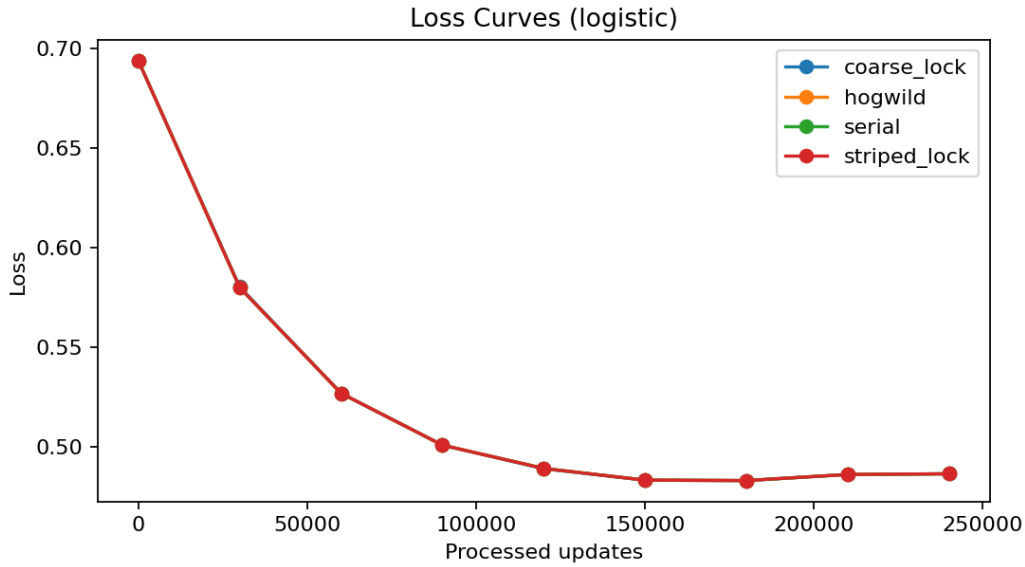

Figure 5: Logistic workload: convergence as a function of update count.

Interpretation: plotting by updates instead of time separates optimization progress from systems overhead.

Comparison takeaway: when curves are similar in update-space but differ in time-space, the main gap comes from implementation/runtime efficiency rather than learning quality.

3.6 Logistic: Performance vs Sparsity

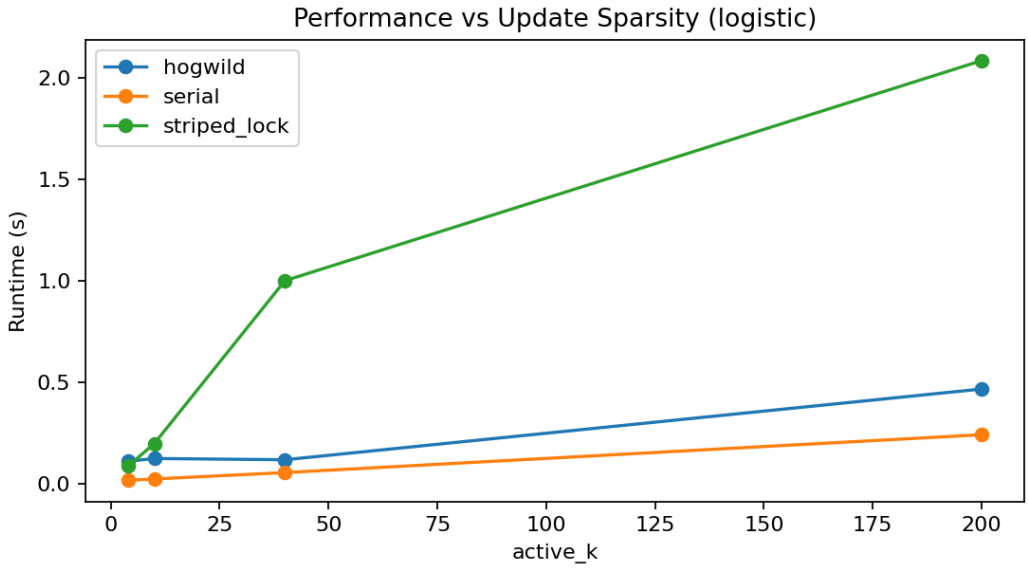

Figure 6: Logistic workload: runtime trend as active features per update (k) increase.

Interpretation: denser updates (larger k) increase work and conflict opportunity.

Comparison takeaway: `hogwild` stays clearly better than `striped_lock` across this sweep, but `serial` still wins overall in this specific budget. This suggests lock-free helps relative to locks, yet not always enough to beat single-thread when jobs are short.

3.7 Logistic: Performance vs Contention

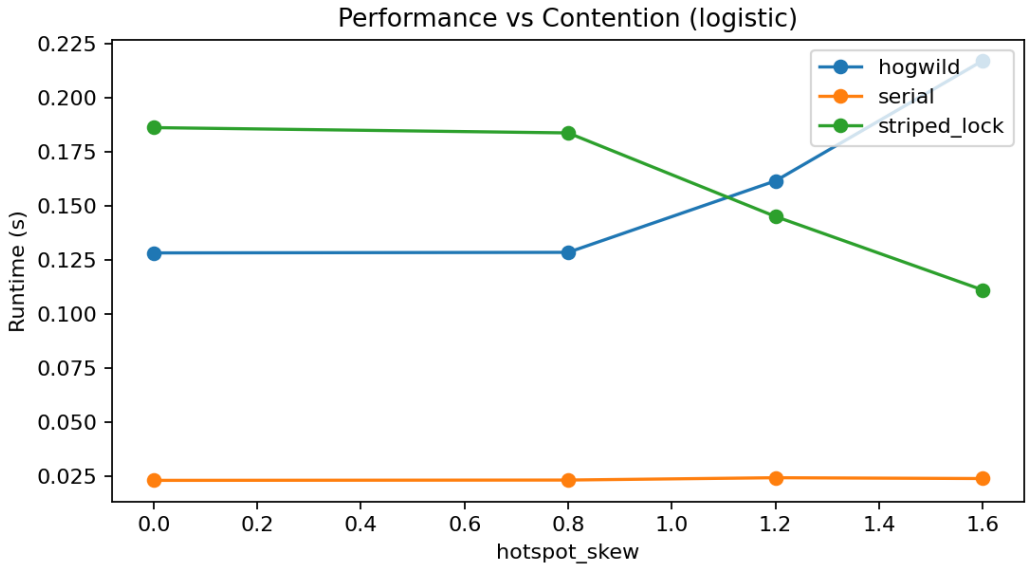

Figure 7: Logistic workload: runtime under increasing hotspot skew/contention.

Interpretation: this figure shows the clearest crossover behavior. In low-contention settings, Hogwild is faster than striped locking. Under extreme contention, lock-free updates degrade and can become slower.

Comparison takeaway: this is the strongest evidence that Hogwild is *condition-dependent*, not universally dominant. It works best when overlap probability is low.

3.8 Logistic: Performance vs Model Size

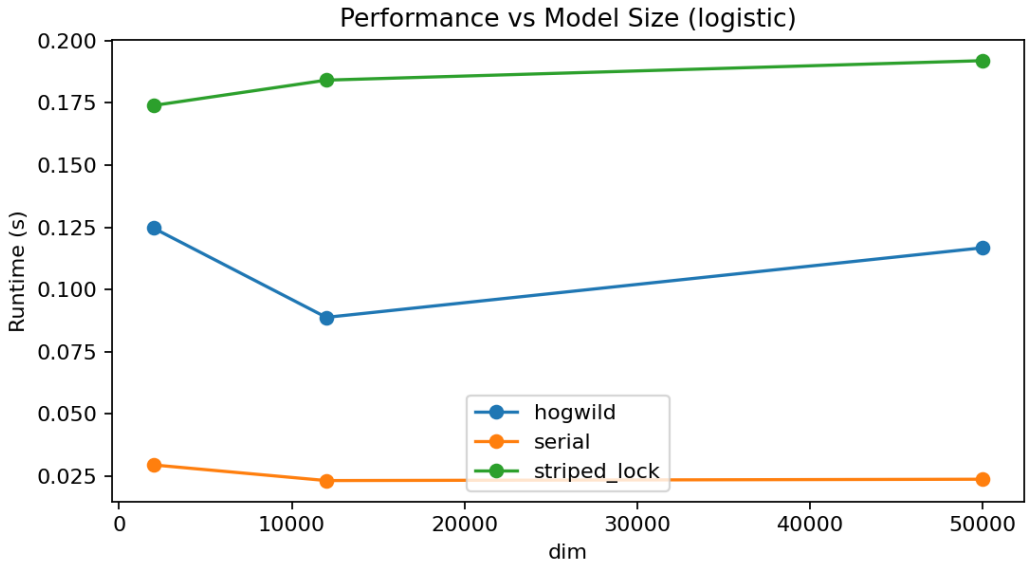

Figure 8: Logistic workload: runtime versus model dimensionality d.

Interpretation: larger models change memory behavior and can alter collision probability at fixed update size.

Comparison takeaway: in this sweep, `hogwild` remains consistently better than striped locking, but model size alone does not force `serial` to lose for the tested epoch budget.

3.9 Matrix Factorization: Time-to-Target vs Threads

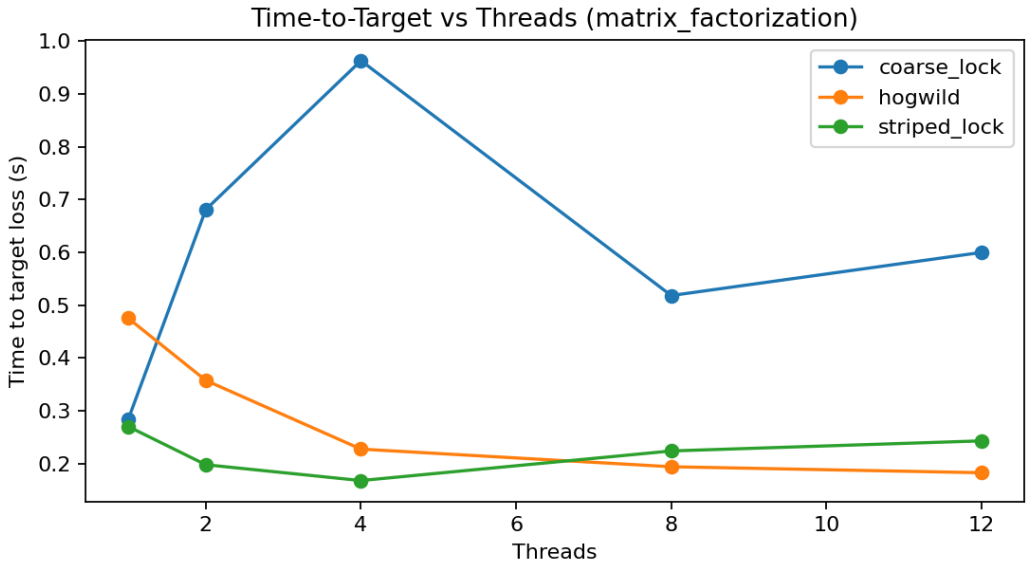

Figure 9: Matrix factorization workload: time-to-target-loss vs thread count.

Interpretation: MF provides a heavier compute pattern than the short logistic runs, so parallelism has more room to pay off.

Comparison takeaway: lock-free and striped-lock strategies can both produce real wall-clock improvements over serial, depending on thread count.

3.10 Matrix Factorization: Throughput vs Threads

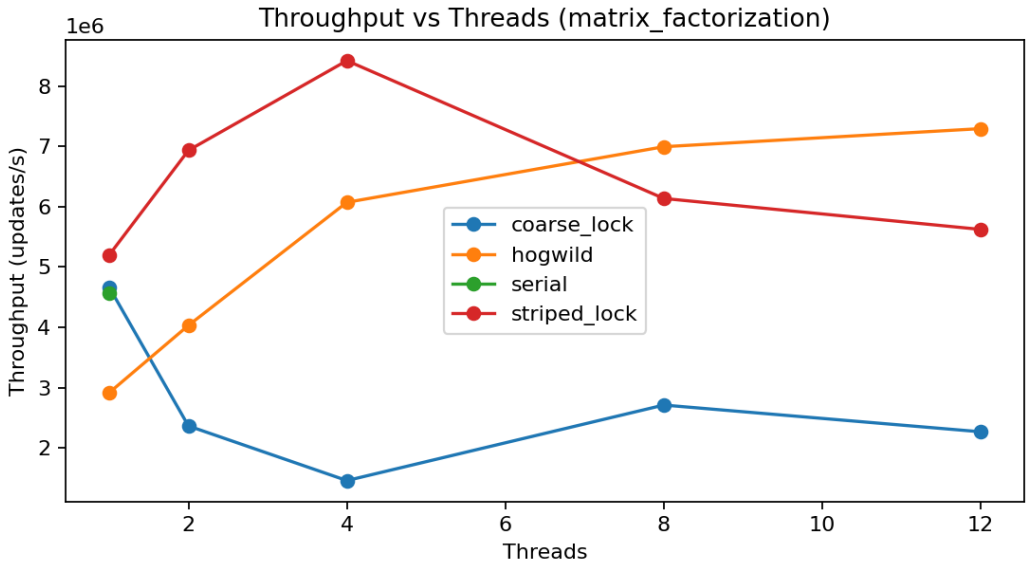

Figure 10: Matrix factorization workload: update throughput vs thread count.

Interpretation: MF throughput scales better with thread count than the lightweight logistic case, because there is more computation per update and less overhead dominance.

Comparison takeaway: this is where multicore execution starts to show clearer practical value.

3.11 Matrix Factorization: Speedup vs Threads

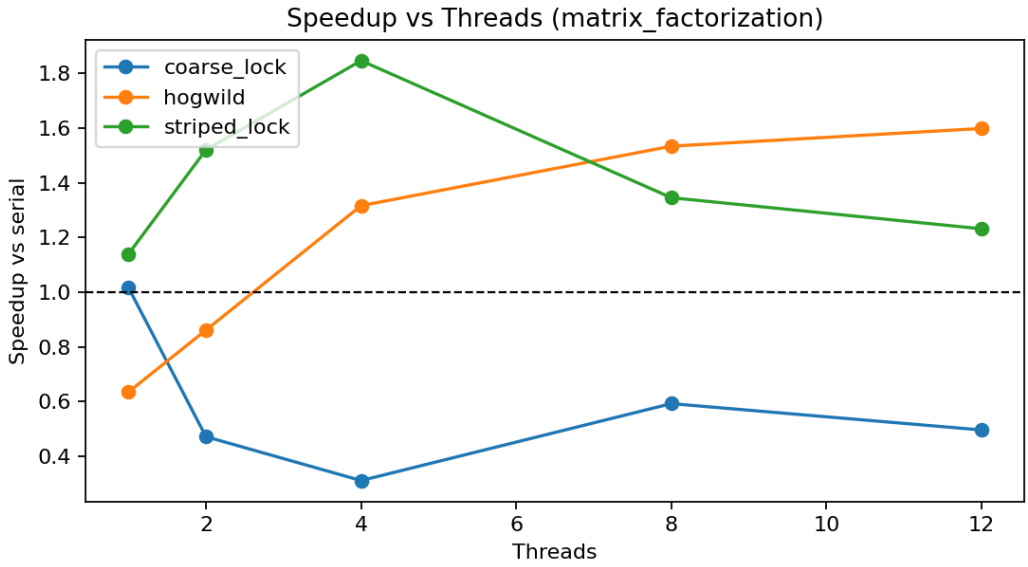

Figure 11: Matrix factorization workload: speedup relative to serial baseline.

Interpretation: MF includes thread settings where speedup exceeds 1, i.e., real end-to-end benefit from parallel training.

Comparison takeaway: unlike short logistic runs, this workload shows that lock-free parallel SGD can beat serial in practice on this machine.

3.12 Matrix Factorization: Loss vs Time

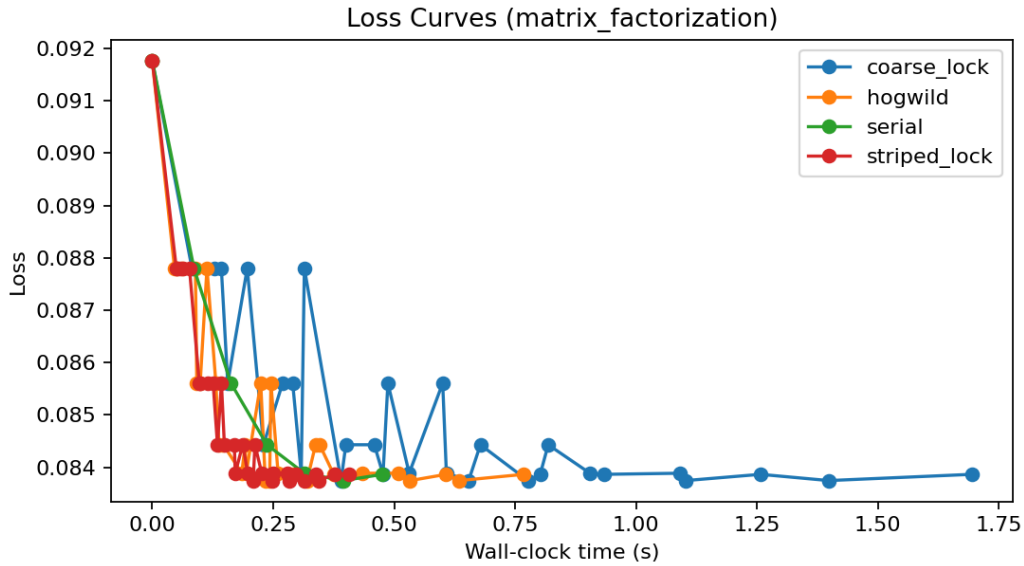

Figure 12: Matrix factorization workload: convergence trajectories over time.

Interpretation: MF curves indicate that parallel methods can maintain useful convergence while reducing wall-clock time.

Comparison takeaway: the optimization remains stable enough while systems-level parallel gains become visible.

4. What the Project Reveals

Across all figures, a consistent story appears:

- **Hogwild is strongest in sparse/low-contention settings**, where lock overhead is mostly avoidable and update collisions are rare.
- **High contention hurts lock-free methods**. As overlap rises, conflicting/stale updates and atomic pressure reduce Hogwild's advantage and can reverse rankings.
- **Workload scale matters**. In very short runs, serial overhead is minimal and parallel launch/synchronization costs dominate; in heavier workloads (especially MF and heavy logistic variants), parallel methods produce real speedups.
- **Synchronized methods are not always bad**. Striped locking remains competitive in some regions, especially when lock-free contention becomes severe.

The key conclusion is not "Hogwild always wins," but rather that **the right synchronization strategy depends on sparsity, contention, and workload granularity**. This aligns with the project hypothesis and with the original Hogwild intuition.

5. Short Limitations Note

This implementation does not yet include thread pinning/affinity control, and contention proxies are empirical rather than exact theoretical constants. So crossover points are hardware- and implementation-sensitive, but the qualitative trends are clear and reproducible.